

Functional interfaces

Saturday, January 24, 2026 5:35 PM

Important functional interfaces and abstract method in it:

=====

- 1] **Predicate<T>**
boolean test(T t);
- 2] **Function<T, R>**
R apply(T t);
- 3] **Consumer<T>**
void accept(T t);
- 4] **Supplier<T>**
T get();
- 5] **BiFunction<T,U,R>**
R apply(T t, U u);

Understanding functional interfaces is essential because lambdas and method references are based on functional interfaces and used in streams.

1. Predicate<T>

=====

Purpose:

-> It is used to Represent a condition or test on an input — returns a boolean result.

-> we use it for filtering purpose in streams.

-> Method:

boolean test(T t);

-> Example:

=====

```
package lambda.interfaces;

import java.util.function.Predicate;

public class DemoPredi
{
    public static void main(String[] args)
    {
        Predicate<Integer> p=t->t%2==0;
        System.out.println(p.test(20));

        Predicate<String> p2=t->t.length()>=5;

        System.out.println(p2.test("programming")); //true
        System.out.println(p2.test("pro")); //false
    }
}
```

```
}
```

```
}
```

2. Function<T, R>

=====

Purpose:

->Represents a function that takes one input of type T and returns a result of type R.

->it is used to transform the data from one form into other form in streams.

Method:

R apply(T t);

ex:

```
package lambda.interfaces;
import java.util.function.Function;
import java.util.function.Predicate;
public class DemoFunction
{
    public static void main(String[] args)
    {
        Function<String, Integer> f1=x->x.length();
        System.out.println(f1.apply("hello"));
        System.out.println(f1.apply("abcdef"));
    }
}
```

o/p

5

6

3. Consumer<T>

=====

Purpose:

->Represents an operation that accepts a single input argument and returns no result.

->Typically used for operations like printing, modifying, etc. in streams, where no return value is needed.

-> Method:

void accept(T t);

```
package lambda.interfaces;
```

```
import java.util.function.Consumer;
```

```
public class DemoConsumer
```

```
{
```

```
    public static void main(String[] args)
    {
```

```

Consumer<String> c=x->{
    System.out.println("hello "+x);
};

c.accept("hemanth");
c.accept("soham");
c.accept("salaar");
}
}

```

4. Supplier<T>

=====

->Purpose:

->Supplier<T> represents a function that does not take any input, but produces (supplies) a result of type T.

->It's useful when you want to generate or provide values which are required in streams.

-> Method:

T get();

ex:

```

package lambda.interfaces;

import java.util.function.Supplier;

public class DemoSupplier
{
    public static void main(String[] args)
    {
        Supplier<String> s=()->
        "hello every one when will you leave jspiders or";

        System.out.println(s.get());
        System.out.println(s.get());
        System.out.println(s.get());

    }
}

```

5. BiFunction<T, U, R>

-> Purpose:

BiFunction<T, U, R> represents a function that:

Takes two input arguments of types T and U

Returns a result of type R

It is useful when you need to combine or transform two values into one in streams.

-> Method:

R apply(T t, U u);

ex:

```
package lambda.interfaces;
```

```
import java.util.function.BiFunction;
```

```
public class DemoBIfun
```

```
{
```

```
    public static void main(String[] args) {  
        BiFunction<Integer, Integer, Integer> bi=(x,y)->x+y;
```

```
        System.out.println(bi.apply(10, 20));
```

```
        System.out.println(bi.apply(20, 30));
```

```
        System.out.println(bi.apply(40, 20));
```

```
    }
```

```
}
```